$\mathbf{EFX_0}$ Allocations with at Most Three Relevant Goods

Evan Lin

University of Illinois Urbana-Champaign

Abstract. An allocation of indivisible goods is EFX_0 if no agent envies another agent's bundle after removing any single good from it, even a good that the envious agent values at zero. We prove that every instance with nonnegative additive valuations in which each agent positively values at most three goods has a complete EFX_0 allocation in which all bundles but at most one have at most two goods. We also give an algorithm, Draft and Exchange (DE), that computes such an allocation in time $O(n(n+m))$ with at most $4n$ trades. After an initial draft, goods are traded along a chain or a ring of agents, each of which gets a higher-ranked holding and loses no value, until a free agent can take the leftover goods, handing at most one to each other free agent (or every agent holds its pair). The main observation is that when no free agent can take them, every agent that does not yet hold its pair can point to an agent that would accept its good in a trade, and these pointers close a ring. The proof is machine-checked in Lean, as are the algorithm, its bound of $4n$ trades and the running time of an implementation.

Keywords: Fair division · Indivisible goods · EFX · Trading cycles

1 Introduction

Let N be a set of $n \geq 1$ agents that share a set M of m indivisible goods and have nonnegative additive valuations $v_i(S) = \sum_{g \in S} v_i(g)$. An allocation X assigns every good to exactly one agent, so no good is left out (allocations are *complete*), and X_i is the bundle of agent i. It is *EFX*$_0$ if

$$v_i(X_i) \;\geq\; v_i(X_j \setminus \{h\}) \qquad \text{for all } i \neq j \text{ and all } h \in X_j,$$

while *EFX* requires this only for goods h with $v_i(h) > 0$ [3]. Thus EFX_0 implies EFX, and the two notions coincide when all values are positive. Under EFX_0, if X_j contains a good that i values at 0, then i must not envy X_j at all. Whether complete EFX allocations always exist for additive valuations is open, although they exist in special cases [5,7,12,2,4,1,6,13,8,9] (Table 1 in Appendix E). Let $R_i = \{g : v_i(g) > 0\}$ be the set of *relevant* goods of i. If $|R_i| \leq 2$ for all i, serial dictatorship gives an EFX_0 allocation [11]. With three goods this fails: an agent that values a, b, c at 4,

3, 2 may take a and later find b and c in the last agent's bundle with a third good, and removing that third good leaves b and c, worth $5 > 4$.

Theorem 1 (existence and shape). *Every instance with $n \geq 1$ agents and nonnegative additive valuations in which every agent positively values at most three goods has a complete EFX$_0$ allocation in which all bundles but at most one have at most two goods.*

Theorem 2 (algorithm). *Algorithm* DE *(Section 6) computes such an allocation with at most n peeling rounds and at most $4n$ trades, in $O(n(n+m))$ steps of a unit-cost random-access machine. Every decision of* DE *compares two sums of at most two values of one agent, and each such comparison counts as one step.*

What is new. As far as we know,[1] no earlier result settles the case of three goods. Viswanathan and Mehta [14] study the same kind of restriction, with up to four relevant goods per agent, but for ordinary EFX and with only a proof sketch. Their result does not imply ours, because EFX does not imply EFX$_0$, and the difference matters precisely when agents value few goods, so that most goods are worth 0 to most agents. In the example above, suppose the agent holds a, worth 4, and another bundle is $\{b, c, g\}$ with $v(g) = 0$. Removing b or c leaves 2 or 3, so EFX holds, but removing g leaves $5 > 4$, so EFX$_0$ fails. Their algorithm gives the goods that nobody values to an arbitrary agent, which can create such a bundle. None of the existence results for complete EFX$_0$ allocations surveyed in Table 1 covers all instances with at most three relevant goods per agent. Apart from [11], which allows at most two relevant goods per agent, each bounds a quantity that our model leaves free: the number of agents, the number of goods compared with agents, the number of agents valuing a good, the girth of the hypergraph of goods, or the number of distinct valuations. Appendix E gives an instance with six agents that none of them covers and checks it row by row. Our contributions are the existence result (Theorem 1), the algorithm (Theorem 2), a machine-checked proof (Appendix F lists what is checked), and the shape of the allocation, which has no counterpart we know of and cannot be strengthened in two natural ways (Appendix D). Some instances have no EFX$_0$ allocation with all bundles of at most two goods, and on an instance with four agents every EFX$_0$ allocation has bundle sizes $4, 1, 1, 1$, so the large bundle cannot be limited to three goods.

[1] Literature search of 25 September 2026 (arXiv, the web, Zenodo, Google Scholar, Crossref, the EC 2026 programme), recorded in `proofs/novelty.md` of [10].

How the proof works. Agents that can leave with at most one good do so, and the others draft at most one good each. Then, until a *free* agent can take the leftover goods, handing at most one to each other free agent (or every agent holds its pair), goods are traded along a *chain* or a *ring* of agents, each of which gets a higher-ranked holding (its pair ranks above its top) and loses no value. Progress is measured by the total score, the sum of these ranks, and not by welfare, because an agent moving between two goods of equal value gains score but no value. Section 6 gives a small example.

2 Peeling

The proofs of Sections 2 and 3 are in Appendix A.

Agents and goods are numbered, and "index order" and "the first" refer to these numbers. A good is *alone* in an allocation if it is the only good of its bundle.

Lemma 1 (peeling). *Let $i \in N$ and $P \subseteq M$ with (i) $v_i(P) \geq v_i(M \setminus P)$ and (ii) $|P| \leq 1$. If the instance with agents $N \setminus \{i\}$ and goods $M \setminus P$ has an EFX$_0$ allocation Z, then Z together with $X_i = P$ is an EFX$_0$ allocation of the whole instance.*

Rule R1. Let G be the set of remaining goods, and let p be agent i's favourite good in G, the one of largest value (the lower index breaks ties). If $G = \emptyset$ or $v_i(p) = 0$, so that i values no remaining good, then i may leave with $P = \emptyset$. If $v_i(p) > 0$ and $v_i(G \setminus \{p\}) \leq v_i(p)$, then i may leave with $P = \{p\}$. Rule R1 applies, for instance, to every agent that values at most two remaining goods. Hence, if every agent values at most two goods, serial dictatorship is EFX$_0$ [11].

When $|R_i| = 3$ we name agent i's relevant goods a_i, b_i, c_i so that $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$, breaking ties in favour of the lower index. This gives a strict *ranking* $a_i \succ_i b_i \succ_i c_i$, and $g \succ_i h$ implies $v_i(g) \geq v_i(h)$. Agent i is *strictly balanced* if $v_i(a_i) < v_i(b_i) + v_i(c_i)$.

Lemma 2 (when nobody can leave). *Let agent i value at most three goods of the set G of remaining goods, and suppose that rule R1 does not apply to i. Then i values exactly three goods of G and is strictly balanced with respect to them.*

Throughout Sections 3–5 we work in the *core case*, in which every agent values exactly three goods and is strictly balanced; there may also be goods that no agent values.

3 States

Definition 1 (states and scores). *A* state Y *gives each agent* i *a* holding Y_i*: nothing,* $\{a_i\}$*,* $\{b_i\}$*,* $\{c_i\}$*, or its* pair $\{b_i, c_i\}$*, with the holdings pairwise disjoint.* U *is the set of agents holding their pair. The goods held by nobody are* leftover*;* J *is the set of leftover goods. Agent* i*'s* score *is* 4*,* 3*,* 2*,* 1 *or* 0 *if it holds its pair,* a_i*,* b_i*,* c_i *or nothing. Since* $v_i(a_i) < v_i(b_i)+v_i(c_i)$*, a higher score never means a lower value.*

A higher score need not mean a higher value, however: if $v_i(a_i) = v_i(b_i)$, moving from b_i to a_i raises the score but not the value. Gaining the pair, or gaining a good after holding nothing, does raise the value. The trades below raise scores, and the total score $\Phi(Y) = \sum_i \text{score}_i(Y)$, an integer between 0 and $4n$, measures progress.

Definition 2 (wants, validity, free agents). *Let* Y *be a state. An agent* $i \notin U$ wants *a good* $g \in R_i$ *if it holds nothing, or holds a good* y *with* $g \succ_i y$*. Agents in* U *want nothing.* Y *is* valid *if every good that some agent wants is held alone: it is the whole holding of some agent. An agent* $i \notin U$ *is* free *if it holds nothing or holds a good that nobody wants.* F *is the set of free agents.*

No agent wants its own good, so the holder of a wanted good is some other agent; in a valid state, this holder is neither in U nor free. Leftover goods and the goods of pair holders are not wanted in a valid state, and an agent outside $U \cup F$ holds a single good that some agent wants.

Validity is what makes finishing possible. As it keeps every wanted good alone, a bundle of two or more goods can contain only goods that nobody wants. An agent holding nothing wants all its goods, and an agent holding a single good wants every good it ranks higher. Such a bundle therefore contains none of the relevant goods of an agent holding nothing, and, of the relevant goods of an agent holding a single good, only goods worth at most that good. One obstruction remains, which arises only with three goods: an agent holding its top a_x may find b_x and c_x together in a bundle with a third good, while $v_x(b_x) + v_x(c_x) > v_x(a_x)$. Section 4 deals with this case.

Lemma 3 (the draft). *Let the agents, in any order, each take the first of* a_i*,* b_i*,* c_i *not yet taken, or nothing if all three are taken. The result is a valid state with* $U = \emptyset$*.*

Lemma 4 (staying valid). *Let* Y *be valid, and* Y' *a state in which every agent's score is at least its score in* Y *and every good wanted in* Y *is held alone. Then* Y' *is valid.*

4 Finishing

Definition 3 (blockers, finishing). *Let Y be a state and o a free agent. A* blocker *of o is an agent $x \neq o$ that holds exactly its top, $Y_x = \{a_x\}$, and has $b_x, c_x \in J \cup Y_o$. Agent o* can finish *with a set $H \subseteq J$ if $|H| \leq |F| - 1$ and H contains b_x or c_x for every blocker x of o. Its* completion *X gives each good of H to a different free agent other than o, gives o the bundle $X_o = Y_o \cup (J \setminus H)$, and lets every other agent keep its holding (plus its good of H, if any).*

Informally, a blocker x of o would find both b_x and c_x in o's bundle if o took all the leftover goods, and the set H moves one of the two out of that bundle. When every agent holds its pair, the completion with any agent o gives o the bundle $Y_o \cup J$ and lets the others keep their pairs.

Theorem 3 (soundness). *Let Y be a valid state, and X the completion with a free agent o that can finish with a set H, or, if every agent holds its pair, with any agent o. Then X is an EFX$_0$ allocation, and every bundle other than X_o has at most two goods.*

Proof. Every bundle other than X_o has at most two goods: a free agent other than o holds at most one good and receives at most one good of H, a pair holder holds two goods, and every other agent keeps a single good.

Wanted goods stay alone: if g is wanted and held alone by j, then j is not free, so $j \neq o$, j receives no good of H, and $X_j = \{g\}$. Hence every good in a bundle of two or more goods is unwanted.

It remains to rule out envy. Let $i \neq j$ and $h \in X_j$; we show that $v_i(X_j \setminus \{h\}) \leq v_i(X_i)$, where $v_i(X_i) \geq v_i(Y_i)$. If $|X_j| \leq 1$, then $X_j \setminus \{h\} = \emptyset$. So let $|X_j| \geq 2$, and let $T = X_j \cap R_i$; it contains no good that i wants and no good of i's holding.

- If i holds its pair, then $T \subseteq \{a_i\}$, and $v_i(T) \leq v_i(a_i) < v_i(b_i) + v_i(c_i)$.
- If i holds nothing, it wants all of R_i, so $T = \emptyset$.
- If $Y_i = \{y\}$, every good of T is ranked below y and worth at most $v_i(y)$. If $|T| \leq 1$, then $v_i(X_j \setminus \{h\}) \leq v_i(T) \leq v_i(y)$. If $|T| = 2$, then $y = a_i$ and $T = \{b_i, c_i\}$. If $X_j = \{b_i, c_i\}$, removing h leaves one good, worth at most $v_i(b_i) \leq v_i(a_i)$. Otherwise $|X_j| \geq 3$, so $j = o$ by the first step; since $b_i, c_i \in X_o \subseteq Y_o \cup J$, agent i is a blocker of o. Then H contains b_i or c_i, which went to an agent other than o, a contradiction. (When every agent holds its pair, only the first case occurs.) ◻

5 Improving

Definition 4 (rings). *Let Y be a state. A* ring *is a cycle $w_0 \to w_1 \to \cdots \to w_{k-1} \to w_0$ of $k \geq 2$ distinct agents outside U, each holding a single good $y_t = y_{w_t}$, such that each arrow $w_t \to w_{t+1}$ (indices modulo k) is*

- *a* want arrow*: w_t wants y_{t+1}; or*
- *a* pair arrow*: w_{t+1} is free, w_t is a blocker of w_{t+1}, and the pair of w_t is $\{y_{t+1}, h_t\}$ with $h_t \in J$;*

and the leftover goods h_t of the pair arrows are pairwise distinct. Trading along the ring *gives each w_t the good y_{t+1}, together with h_t for a pair arrow, in place of its own good; agents off the ring keep their holdings.*

As in envy graphs, an arrow points from an agent to the holder of a good it wants, or from a blocker to the free agent whose good completes its pair. Goods move against the arrows: along $w_t \to w_{t+1}$, the good of w_{t+1} goes to w_t. In a valid state, an arrow into a free agent is a pair arrow, because nobody wants its good, and an arrow into any other agent is a want arrow.

Lemma 5 (ring). *Trading along a ring of a valid state gives a valid state in which every agent of the ring has a higher score and every other agent the same score.*

Proof. The result is a state: each good y_{t+1} moves to w_t only, the goods h_t are distinct leftover goods, and agents off the ring keep goods that are neither. Each new holding is a single good of its receiver or the receiver's pair. Scores rise along the ring, since along a want arrow w_t receives a good it ranks above what it held, and along a pair arrow w_t goes from its top to its pair. Finally, let g be wanted in Y, held alone by j. If j is off the ring, it keeps g. If $j = w_{t+1}$, then w_{t+1} is not free, so the arrow into it is a want arrow, and w_t now holds g alone. Lemma 4 gives validity. □

Lemma 6 (chain). *Let Y be valid, and x an agent with $Y_x = \{a_x\}$ and $b_x, c_x \in J$. Let $j_0 = x$, and as long as j_t holds a good that some agent wants, let j_{t+1} be the first such agent. If an agent repeats, the agents from its first occurrence on form a ring of want arrows. Otherwise the sequence stops at an agent j_k ($k \geq 0$) that holds nothing or a good nobody wants, and the following gives a valid state in which $x, j_1, \ldots, j_k$ have higher scores and every other agent the same score: x takes its pair; each j_t with $1 \leq t \leq k$ takes the good of j_{t-1}; the old good of j_k, if any, becomes leftover.*

Proof. Agent x holds its top and wants nothing, so $j_t \neq x$ for $t \geq 1$. If $j_s = j_t$ with $1 \leq s < t$ is the first repeat, then $j_s, \ldots, j_{t-1}$ are distinct agents outside U, each holding a good that the next one wants (and $j_s = j_t$ wants the good of j_{t-1}), so $j_{t-1} \to j_{t-2} \to \cdots \to j_s \to j_{t-1}$ is a ring of want arrows ($t - s \geq 2$, as no agent wants its own good). Otherwise the result is a state, since b_x, c_x were leftover and each good that moves goes one step. Agent x goes from its top to its pair, and each j_t, $t \geq 1$, receives a good it wants. A wanted good is neither b_x nor c_x, which are leftover. If its holder is off the path, it keeps the good; if its holder is j_t, then $t < k$ (the good of j_k is not wanted), and j_{t+1} now holds it alone. Lemma 4 gives validity. □

Applying a chain or trading along a ring is called a *trade*. By Lemmas 5 and 6, a trade keeps the state valid and raises the total score Φ by at least one. From now on we assume that no chain applies:

(NC) no agent x with $Y_x = \{a_x\}$ has both b_x and c_x in J.

Lemma 7 (the finishing test). *Let Y be a state with (NC), and o a free agent.*

(a) If o holds nothing, it has no blocker.
(b) If x is a blocker of o, then o holds a good y_o, and the pair of x is $\{y_o, h_x\}$ for a leftover good h_x.
(c) Let H_o be the set of the goods h_x of the blockers x of o. Then o can finish if and only if $|H_o| \leq |F| - 1$, and then it can finish with H_o. In particular a free agent holding nothing can finish.

Proof. Let x be a blocker of o. Then $b_x, c_x \in J \cup Y_o$, and by (NC) not both are in J, so one of them is in Y_o, which proves (a). As o holds at most one good, that good is y_o, and the other one, h_x, is leftover, which proves (b). A set $H \subseteq J$ contains b_x or c_x if and only if it contains h_x, so the sets H allowed by Definition 3 are those with $H_o \subseteq H \subseteq J$ and $|H| \leq |F| - 1$, which exist if and only if $|H_o| \leq |F| - 1$. If o holds nothing, then $H_o = \emptyset$ and $|F| \geq 1$. □

Theorem 4 (Improvement Lemma). *Let Y be a valid state in which not every agent holds its pair and no free agent can finish. Then a chain or a ring gives a valid state in which every agent has at least its score and some agent a higher one.*

Proof. If (NC) fails, Lemma 6 applies, so assume (NC). By Lemma 7, every free agent o then holds a good and has $|H_o| \geq |F|$.

We first choose distinct leftover goods. List the free agents in index order as $o_1, \ldots, o_f$. For $i = 1, \ldots, f$ in turn, let x_{o_i} be the first blocker of o_i whose leftover good is not among $q_{o_1}, \ldots, q_{o_{i-1}}$, and let $q_{o_i} = h_{x_{o_i}}$. This blocker exists because the blockers of o_i have $|H_{o_i}| \geq f > i - 1$ different leftover goods.

Next, for every agent $j \notin U$ let $\sigma(j) = x_j$ if j is free; otherwise let $\sigma(j)$ be the first agent that wants the good of j, which exists because $j \notin U \cup F$. In both cases $\sigma(j) \notin U$, $\sigma(j) \neq j$, and $\sigma(j) \to j$ is an arrow: a pair arrow if j is free, and a want arrow otherwise. So $\sigma(j)$ is the agent that would receive the good of j, and following σ follows the goods, against the arrows.

To find the ring, start at the first agent outside U and follow σ until an agent repeats. The agents from its first occurrence on, taken in reverse order, form a cycle $w_0 \to \cdots \to w_{k-1} \to w_0$ with $\sigma(w_{t+1}) = w_t$ and $k \geq 2$, each holding a single good (for free agents, by Lemma 7). By Lemma 7(b), the arrow into a free agent w_{t+1} is a pair arrow with $h_t = q_{w_{t+1}}$, and these goods are distinct for distinct free agents. Every other arrow is a want arrow. Hence the cycle is a ring, and Lemma 5 applies. □

6 The Algorithm Draft and Exchange

Algorithm 1 puts these pieces together; ties are broken by index as in Section 2.

Theorem 5 (DE is correct). *On every instance in which every agent values at most three goods,* DE *returns a complete EFX$_0$ allocation in which all bundles but at most one have at most two goods. It peels at most n agents and makes at most $4n$ trades.*

Proof. Each peeling round removes one agent, so there are at most n rounds. For the agent i peeled with P_i, rule R1 amounts to conditions (i) and (ii) of Lemma 1 on the instance left before its round, so by induction from the last round down it suffices that DE returns an EFX$_0$ allocation of the agents A and goods G left at the end. If $|A| = 1$, the single agent has no other bundle to envy.

Now let $|A| \geq 2$. Rule R1 applies to no agent of A, so by Lemma 2 we are in the core case. The draft gives a valid state (Lemma 3). A chain or a ring gives a valid state in which no score drops and some score rises (Lemmas 6 and 5); by the proof of Theorem 4, the ring exists whenever it is used. The total score Φ is an integer between 0 and $4|A|$ and rises by at least one with every trade, so there are at most $4|A| \leq 4n$ trades, and

the loop stops. When it stops, either every agent holds its pair or o can finish with H (by Lemma 7(c), since the failed chain test gives (NC)), and Theorem 3 gives an EFX$_0$ allocation in which only X_o can have more than two goods. Peeled agents receive at most one good each. □

A small example. Agents z, x, o, in index order, rank $g_4 \succ g_0 \succ g_1$, $g_0 \succ g_1 \succ g_2$ and $g_0 \succ g_4 \succ g_1$, respectively, with values 4, 3, 2; nobody values g_3. No agent is peeled. In the draft z takes g_4, x takes g_0 and o takes g_1, so the leftover goods are g_2 and g_3. Agent o wants g_0 and g_4, which are held alone, so the state is valid, and o is the only free agent. No chain applies. Agent x is a blocker of o: if o took $\{g_1, g_2, g_3\}$, removing g_3 would leave g_1 and g_2, worth $5 > 4$ to x. Thus $H_o = \{g_2\}$, and as no other free agent can take g_2, o cannot finish. The pointers are $\sigma(o) = x$, for the pair arrow $x \to o$ (x's pair is o's good g_1 and the leftover g_2), and $\sigma(x) = \sigma(z) = o$, for the want arrows $o \to x$ and $o \to z$ (o wants g_0 and g_4). Following σ from z reaches o and then x, which points back to o: the ring is $x \to o \to x$, and along it x takes $\{g_1, g_2\}$ and o takes g_0. Now nobody wants anything, the free agents are z and o, and z has no blocker, so z finishes, and DE returns the EFX$_0$ allocation $X_z = \{g_4, g_3\}$, $X_x = \{g_1, g_2\}$, $X_o = \{g_0\}$.

Algorithm 1 Draft and Exchange (DE)

Require: agents N, goods M, nonnegative values $v_i(g)$, every agent with $|R_i| \le 3$

$A \leftarrow N$, $G \leftarrow M$
while $|A| \ge 2$ and rule R1 applies to an agent of A, for G **do** ▷ peeling
  $i \leftarrow$ the first such agent; $X_i \leftarrow P_i$ (by rule R1); $A \leftarrow A \setminus \{i\}$, $G \leftarrow G \setminus P_i$
if $|A| = 1$ **then** give G to the agent of A and **return** X
for $i \in A$ in index order **do** ▷ the draft
  $Y_i \leftarrow$ the first of $\{a_i\}, \{b_i\}, \{c_i\}$ whose good is not taken, else nothing
loop
  if some x with $Y_x = \{a_x\}$ has $b_x, c_x \in J$ **then** ▷ chain
    apply the chain of the first such x, or the ring of want arrows it runs into (Lemma 6)
  else if every agent holds its pair **then** ▷ finish
    $o \leftarrow$ the first agent; $H \leftarrow \emptyset$; **break**
  else if some free agent o has $|H_o| \le |F| - 1$ **then** ▷ finish
    $o \leftarrow$ the first such agent; $H \leftarrow H_o$; **break**
  else ▷ ring
    trade along the ring of the proof of Theorem 4
give the k-th good of H (in the order of first blockers) to the k-th free agent other than o
$X_o \leftarrow Y_o \cup (J \setminus H)$; every other $i \in A$ keeps Y_i (plus its good of H, if any)
return X

Running time. We count steps of a unit-cost random-access machine, in which an array access, an arithmetic operation on indices, and a comparison of two sums of at most two values of one agent each take one step. This is not a bound on bit complexity for arbitrary encodings of the values. DE keeps an array of the holder of each good (or a mark that the good is leftover or peeled) and an array of the holding of each agent. One pass over the nm values finds the at most three relevant goods of each agent, and sorting them gives its ranking, in $O(nm)$ steps. To test rule R1 for agent i, we look only at its relevant goods still in G, find its favourite p and compare $v_i(p)$ with the sum of the at most two other values; a peeling round thus tests every agent in $O(n)$ steps and removes at most one good.

An iteration of the loop takes $O(n + m)$ steps. Every agent outside U marks the at most three goods it wants, and each good records the first agent, in index order, that wants it. Then one pass finds the leftover goods, another the free agents, and a third runs the chain test, which looks up the holders of b_x and c_x for every x holding a_x. A chain follows the recorded agents, marking those it visits, in $O(n)$ steps. Under (NC), an agent x holding a_x is a blocker of o exactly when one of b_x, c_x is leftover and the other is the good of the free agent o (Lemma 7). Each agent is therefore a blocker of at most one free agent, and one pass in index order lists the blockers of every free agent, each list in index order. One pass over these lists, with marks on goods, gives the sizes $|H_o|$, and another gives the distinct leftover goods q_o of the proof of Theorem 4. Then each $\sigma(j)$ takes one step, and following σ while marking visited agents finds the ring in $O(n)$ steps. A trade and the final completion each take $O(n+m)$ steps. With at most n peeling rounds and at most $4n + 1$ loop iterations (the last one finishes), DE takes $O(nm+n^2+n(n+m)) = O(n(n+m))$ steps. For a counted program, Lean proves a bound of $750(n + 1)(n + m + 1)$ steps (Appendix F).

Real values. Every decision of DE compares one value of an agent with 0, with another of its values, or with the sum of two of its values, and the proofs use nothing else about the values. Theorem 5 and the remarks above therefore prove Theorems 1 and 2, for real values as well. Appendix F explains how the Lean proof handles real values.

7 Further Results, Verification and Open Problems

Appendix B has a larger example, in which DE's ring gives pairs to two agents at once; rings may need any number of pair arrows (Appendix C),

and some instances need a bundle of three or four goods (Appendix D). Every result here is machine-checked in core Lean 4 (`lean/EFX/K3DE*.lean`, with `RealValues.lean` and `K3Real.lean` for real values, in [10]), including the main theorem for natural-number values, its extension to real values through natural numbers that answer the same comparisons, and the running time of a counted program that returns DE's output. Appendix F maps each result to its Lean theorem (Table 2) and says what is not formalized. Tests found no failure. All reviews so far were done by AI agents, and there has been no human peer review yet. Three questions remain open. Does every instance in which each agent positively values at most four goods have a complete EFX_0 allocation? (For ordinary EFX, Viswanathan and Mehta [14] announce a positive answer.) Do fewer than $4n$ trades suffice? What is the least bound on the size of the large bundle that holds for every instance?

Acknowledgments. The proofs, code, Lean formalizations and this paper were developed with the assistance of AI coding agents (Claude Code).

References

1. Afshinmehr, M., Ashuri, A., Mahmoudkhan, P., Mehlhorn, K., Shahrezaei, A.M.: EFX allocations exist on multi-graphs. arXiv preprint arXiv:2606.18665 (2026), accepted at the 27th ACM Conference on Economics and Computation (EC 2026)
2. Alkassar, E., Fouz, M., Mehlhorn, K.: Complete EFX allocations exist for four additive agents and up to nine goods. arXiv preprint arXiv:2608.08590 (2026)
3. Amanatidis, G., Birmpas, G., Filos-Ratsikas, A., Voudouris, A.A.: Fair division of indivisible goods: A survey. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22). pp. 5385–5393 (2022). `https://doi.org/10.24963/ijcai.2022/756`, arXiv:2202.07551
4. Bratby, J.: Complete EFX allocations with four more goods than agents. Zenodo preprint, version 1, `https://doi.org/10.5281/zenodo.22665180`; claims complete EFX_0 allocations when $m = n + 4$; not peer-reviewed (2026)
5. Chaudhury, B.R., Garg, J., Mehlhorn, K.: EFX exists for three agents. Journal of the ACM **71**(1), 4:1–4:27 (2024). `https://doi.org/10.1145/3616009`, conference version in Proc. 21st ACM Conference on Economics and Computation (EC 2020), pp. 1–19; full version arXiv:2002.05119
6. Christodoulou, G., Fiat, A., Koutsoupias, E., Sgouritsa, A.: Envy-freeness up to any good allocations on graphs. Mathematics of Operations Research (2026). `https://doi.org/10.1287/moor.2025.0915`, conference version: "Fair Allocation in Graphs", in Proc. 24th ACM Conference on Economics and Computation (EC 2023), pp. 473–488
7. HV, V.P., Ghosal, P., Nimbhorkar, P., Varma, N.: EFX exists for three types of agents. In: Proceedings of the 26th ACM Conference on Economics and Computation (EC 2025). pp. 101–128. ACM (2025). `https://doi.org/10.1145/3736252.3742509`, arXiv:2410.13580

8. Kaviani, A., Seddighin, M., Shahrezaei, A.: Almost envy-free allocation of indivisible goods: A tale of two valuations. In: Web and Internet Economics (WINE 2024). pp. 261–276. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-08560-3_15, full version arXiv:2407.05139
9. Lianeas, T., Sgouritsa, A., Sotiriou, M.M.: EFX allocation in (multi)hypergraphs. Proceedings of the AAAI Conference on Artificial Intelligence **40**(20), 17102–17110 (2026). https://doi.org/10.1609/aaai.v40i20.38759, AAAI-26; arXiv:2608.03171
10. Lin, E.: efx0-three-goods-proof: EFX_0 with at most three relevant goods per agent (paper, Lean formalization, tests, referee reports). GitHub repository, tag v1.0, https://github.com/evanlin23/efx0-three-goods-proof (2026)
11. Lin, E., Osterhaus, G.: Strong EFX allocations for 2-relevant agents: Lean 4 formalization. GitHub repository mrd-efx, tag v1.2.0 (commit fa42aa1, 23 September 2026), https://github.com/evanlin23/mrd-efx (2026)
12. Mahara, R.: Extension of additive valuations to general valuations on the existence of EFX. Mathematics of Operations Research **49**(2), 1263–1277 (2024). https://doi.org/10.1287/moor.2022.0044, conference version in Proc. 29th Annual European Symposium on Algorithms (ESA 2021), LIPIcs 204, 66:1–66:15; arXiv:2107.09901
13. Sgouritsa, A., Sotiriou, M.M.: On the existence of EFX allocations in multigraphs. arXiv preprint arXiv:2502.09777 (2025), extended abstract, without the bipartite case, in Proc. 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025), pp. 2735–2737
14. Viswanathan, M., Mehta, R.: On the existence of EFX under picky or non-differentiative agents. In: Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024). pp. 2534–2536 (2024). https://doi.org/10.65109/osos1146, extended abstract

A Proofs of Sections 2 and 3, and a Corollary

Proof (of Lemma 1). For agent i and a bundle Z_j, $v_i(Z_j \setminus \{h\}) \leq v_i(Z_j) \leq v_i(M \setminus P) \leq v_i(P)$. For an agent $j \neq i$ and the bundle P, removing its only good leaves nothing, and for $j \neq i$ and a bundle Z_k, the condition holds because Z is EFX_0. □

Proof (of Lemma 2). Since rule R1 does not apply, $v_i(p) > 0$ and $v_i(G \setminus \{p\}) > v_i(p)$. If i valued at most two goods of G, then $v_i(G \setminus \{p\})$ would be the value of at most one good, and hence at most $v_i(p)$. Therefore i values exactly three goods of G; p is its top, and $v_i(b_i) + v_i(c_i) > v_i(a_i)$. □

Proof (of Lemma 3). Each agent holds one of its goods or nothing, and no good is taken twice. If i wants g, then g was taken before i's turn by an earlier agent, as that agent's only good, and holdings never change during the draft. □

Proof (of Lemma 4). Let i want g in Y'. If i holds nothing in Y', its score in Y was 0, so it held nothing in Y and wanted g there too. If i holds a

single good y' in Y', its score in Y was below 4, so in Y it held either nothing or a good y with $y' = y$ or $y' \succ_i y$; since $g \succ_i y'$, again i wanted g in Y. Hence g is held alone in Y'. □

Corollary 1 (score-Pareto-maximal valid states). *In the core case, in every valid state that no valid state Pareto-dominates in scores, either every agent holds its pair or some free agent can finish. In particular every core instance has a complete EFX$_0$ allocation in which all bundles but at most one have at most two goods.*

Proof. The first part follows from Theorem 4. There are valid states (Lemma 3), and only finitely many, so one with the largest total score is not Pareto-dominated in scores, and Theorem 3 applies to it. □

This gives a second existence argument, within the valid states, but not an efficiency guarantee: maximality is in scores and among valid states only, and we claim nothing about the efficiency of the resulting allocation or of DE's output.

B A Worked Example

We run DE on an instance in which every valid state that Pareto-dominates the draft state in scores gives pairs to two agents at once, so a ring with a single pair arrow is not enough (Appendix C). It has ten goods $g_0, \ldots, g_9$ and six agents, in index order x_1, x'_1, x_2, x'_2, o_1, o_2, with values 4, 3, 2 on their goods $a \succ b \succ c$ (any strictly balanced values with these rankings give the same run):

agent	$a \succ b \succ c$	after the draft	after the trade
x_1	$g_0 \succ g_4 \succ g_6$	$\{g_0\}$ (a)	$\{g_4, g_6\}$ (pair)
x'_1	$g_1 \succ g_4 \succ g_7$	$\{g_1\}$ (a)	$\{g_1\}$ (a)
x_2	$g_2 \succ g_5 \succ g_8$	$\{g_2\}$ (a)	$\{g_5, g_8\}$ (pair)
x'_2	$g_3 \succ g_5 \succ g_9$	$\{g_3\}$ (a)	$\{g_3\}$ (a)
o_1	$g_2 \succ g_3 \succ g_4$	$\{g_4\}$ (c)	$\{g_2\}$ (a)
o_2	$g_0 \succ g_1 \succ g_5$	$\{g_5\}$ (c)	$\{g_0\}$ (a)

The draft. No agent is peeled. The four x's take their tops, o_1 finds g_2 and g_3 taken and takes g_4, and o_2 likewise takes g_5. Agent o_1 wants g_2 and g_3, agent o_2 wants g_0 and g_1, and the x's want nothing; all four wanted goods are held alone. The leftover goods are $g_6, \ldots, g_9$, and the free agents are o_1 and o_2. No chain applies, since the b of each x is held by o_1 or o_2.

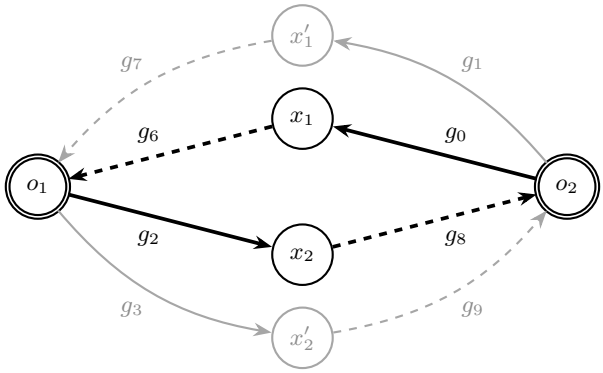


Fig. 1. The arrows of the draft state of Appendix B. Double circles are the free agents. Dashed arrows are pair arrows, from a blocker to the free agent whose good completes its pair, labelled by the blocker's leftover good; solid arrows are want arrows, from an agent to the holder of a good it wants, labelled by that good. Goods move against the arrows. The ring chosen by DE is drawn in black. Every cycle of these arrows has four agents and two pair arrows.

Nobody can finish. The blockers of o_1 are x_1, with pair $\{g_4, g_6\}$, and x'_1, with pair $\{g_4, g_7\}$, so $H_{o_1} = \{g_6, g_7\}$; symmetrically, $H_{o_2} = \{g_8, g_9\}$. Thus $|H_o| = 2 > |F| - 1$ for both. Concretely, if o_1 takes the leftover goods, only o_2 can take one of g_6, g_7. With $H = \{g_6\}$, agent o_1 gets $X_{o_1} = \{g_4, g_7, g_8, g_9\}$, and x'_1, holding g_1 (worth 4), values X_{o_1} without g_8 at $3 + 2 = 5$. All ten candidate completions (with o_1 or o_2 finishing and H empty or a single leftover good) fail the EFX_0 test.

No small trade helps. Every want arrow starts at o_1 or o_2, the only agents that want anything, so a ring with no pair arrow would have to enter one of them by a want arrow, but nobody wants their goods. A ring with a single pair arrow, say into o_1, would have to leave o_1 by a want arrow and return to o_1's blocker by want arrows only; but the want arrows from o_1 end at x_2 and x'_2, which want nothing, and the only arrows from x_2 and x'_2 are pair arrows into o_2. In fact exactly four valid states Pareto-dominate the draft state in scores, and each gives pairs to two agents.

The trade. DE takes the first blocker of each free agent, $x_{o_1} = x_1$ with $q_{o_1} = g_6$ and $x_{o_2} = x_2$ with $q_{o_2} = g_8$. Hence $\sigma(o_1) = x_1$, $\sigma(x_1) = o_2$ (o_2 wants g_0), $\sigma(o_2) = x_2$ and $\sigma(x_2) = o_1$ (o_1 wants g_2). Each $\sigma(j) \to j$ is an arrow, so this is the ring $o_1 \to x_2 \to o_2 \to x_1 \to o_1$ of Figure 1. Along it, x_1 takes $\{g_4, g_6\}$, o_2 takes g_0, x_2 takes $\{g_5, g_8\}$ and o_1 takes g_2, and the total score rises from 14 to 20.

Finishing. Now nobody wants anything, the leftover goods are g_7 and g_9, and the free agents are x_1', x_2', o_1 and o_2. No chain applies, since the b of x_1' is g_4, now held by x_1, and similarly for the others. The first free agent, x_1', has no blocker, because no other agent holding its top has both b and c in $\{g_1, g_7, g_9\}$. So x_1' finishes with $H = \emptyset$, and DE returns

$$X_{x_1} = \{g_4, g_6\}, \quad X_{x_1'} = \{g_1, g_7, g_9\}, \quad X_{x_2} = \{g_5, g_8\},$$
$$X_{x_2'} = \{g_3\}, \quad X_{o_1} = \{g_2\}, \quad X_{o_2} = \{g_0\}.$$

To check this directly, note that the only bundle of three goods, $X_{x_1'}$, is worth 3 to o_2 (from g_1 alone) and 2 to x_2' (from g_9 alone), while o_2 and x_2' hold g_0 and g_3, each worth 4 to its holder. Without one of its goods, a bundle of two goods is worth at most 3 to any other agent, and every agent holds at least 4.

C How Large Must a Ring Be?

A *short move* is a chain, or a ring with at most one pair arrow. A natural, simpler algorithm would use only short moves. The next proposition shows that, among states in which short moves are not enough, the instance of Appendix B has the smallest possible number of agents.

Proposition 1 (when short moves are not enough). *In the core case, let Y be a valid state in which not every agent holds its pair, no free agent can finish, and no short move applies. Then $|F| \geq 2$, $n \geq 6$ and $m \geq 8$; if $|F| \geq 3$, then $n \geq 9$ and $m \geq 12$. The bounds $n = 6$ and $m = 8$ are attained.*

Proof. No chain applies, so (NC) holds, and by Lemma 7 every free agent o holds a good and has $|H_o| \geq |F|$, hence at least $|F|$ blockers. The blockers of two different free agents o, o' are different, since a common blocker would have the pair $\{y_o, y_{o'}\}$, with no leftover good, contrary to Lemma 7(b). There is no ring of want arrows, so a walk that repeatedly moves to an agent that wants the current agent's good, backwards along a want arrow, never repeats an agent; it ends at an agent that holds nothing or a good nobody wants, that is, at a free agent. Every agent holds a good: pair holders hold two, free agents one by Lemma 7, and the others a wanted good. Let $k = |F|$; then $k \geq 1$, because the walk from an agent outside U ends at a free agent.

If $k = 1$, let $F = \{o\}$ and let x be a blocker of o. Then x is not free, so its walk ends at o. The want arrows of the walk lead back from o to x,

and together with the pair arrow $x \to o$ they form a ring with one pair arrow, which is a short move. Hence $k \geq 2$.

If $k = 2$, let $F = \{o_1, o_2\}$. Agent o_1 has at least two blockers and o_2 is the only other free agent, so some blocker x of o_1 is not free. As above, its walk cannot end at o_1, so it ends at o_2, and its last step is a want arrow from o_2 ($x \neq o_2$). Agent o_2 therefore wants a good and does not hold its top, so o_2 is not a blocker of o_1, and no blocker of o_1 is free. The same holds for o_2. Hence $n \geq 2 + 2 + 2 = 6$ and $m \geq n + |J| \geq 6 + 2 = 8$.

If $k \geq 3$, the blockers of the k free agents include at least k^2 different agents, at most k of which are free, so $n \geq |F| + (k^2 - k) = k^2 \geq 9$, and $m \geq n + |J| \geq k^2 + k \geq 12$.

The instance of Appendix B attains $n = 6$ (with $m = 10$). For $m = 8$, let o_1 rank $g_4 \succ g_5 \succ g_0$, o_2 rank $g_6 \succ g_7 \succ g_1$, x_1 rank $g_4 \succ g_1 \succ g_2$, x_1' rank $g_5 \succ g_3 \succ g_1$, x_2 rank $g_6 \succ g_0 \succ g_2$ and x_2' rank $g_7 \succ g_3 \succ g_0$, and let o_1 hold g_0, o_2 hold g_1 and every x its top. Then $J = \{g_2, g_3\}$ and $F = \{o_1, o_2\}$; the blockers of o_2 are x_1, x_1' (leftover goods g_2, g_3) and those of o_1 are x_2, x_2' (leftover goods g_2, g_3). Nobody can finish, and no short move applies. □

In the second instance, distinctness of the leftover goods matters. Of the four cycles $o_2 \to x' \to o_1 \to x \to o_2$ with two pair arrows ($x \in \{x_1, x_1'\}$, $x' \in \{x_2, x_2'\}$), two use the same leftover good twice, and trading along them would give that good to two agents; only the other two are rings.

No bounded number of pair arrows suffices. For every $k \geq 2$ and $d \geq 1$ there are valid states with k free agents $o_0, \ldots, o_{k-1}$ of the following form. Each o_i is the root of a binary tree of depth d whose agents want the goods of their two children, and the leaves of the tree of o_{i+1} hold their tops and are blockers of o_i (indices modulo k). Either each leaf has a leftover good of its own, or the leaves of each tree take the goods of a common pool of k in turn, so that a tree's leaves have $\min(2^d, k)$ distinct leftover goods. In these states every cycle of want and pair arrows has exactly k pair arrows, and no free agent can finish if and only if $k \leq 2^d$, the number of leaves of a tree. For $k = 2$ and $d = 1$ this is the instance of Appendix B. We omit the proofs of these two facts; both are machine-checked in Lean for every k and d.

D Limits of the Shape

Could every instance have an EFX_0 allocation in which all bundles have at most two goods, or at least one in which all bundles have at most

three? In both cases the answer is no. Write $\theta_i(B)$, the *threat* of B to i, for the largest value of $v_i(B \setminus \{h\})$ over $h \in B$, or 0 if $B = \emptyset$. Agent i does not envy X_j in the sense of EFX$_0$ if and only if $v_i(X_i) \geq \theta_i(X_j)$, and i is *safe* if this holds for every $j \neq i$. By additivity, $\theta_i(B) = 0$ if $|B| \leq 1$, $\theta_i(B) \leq v_i(B \cap R_i)$ with equality if B has a good outside R_i, and $\theta_i(\{g, h\}) = \max(v_i(g), v_i(h))$.

Lemma 8 (cases of safety). *Let agent i have $R_i = \{a, b, c\}$ with $v_i(a) > v_i(b) > v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$, and let X be an allocation. Then i is safe in X if and only if one of the following holds:*

(T) *i holds a, and either i also holds b or c, or b and c are not together in a bundle of at least three goods;*
(BC) *i holds b and c;*
(B) *i holds b, and a is alone;*
(C) *i holds c, and a and b are alone;*
(E) *a, b and c are all alone.*

Proof. We also write a, b, c for the values. The sums of subsets of $\{a, b, c\}$ are ordered as $0 < c < b < a < b + c < a + c < a + b < a + b + c$. Let $S = X_i \cap R_i$.

- $S \supseteq \{a, b\}$ or $S \supseteq \{a, c\}$: every other bundle meets R_i in at most one good, so its threat is at most $b < a + c$. This gives (T).
- $S = \{b, c\}$: every threat is at most $a < b + c$. This gives (BC).
- $S = \{a\}$: if b, c are in different bundles, each threat is at most $b < a$; if $\{b, c\}$ is a bundle, its threat is $b < a$; if a bundle strictly contains $\{b, c\}$, it has a third good, outside R_i, and its threat is $b + c > a$. This gives (T).
- $S = \{b\}$: if a is not alone, its bundle has threat at least $a > b$; if a is alone, every threat is at most $c < b$. This gives (B).
- $S = \{c\}$: as before, a must be alone, and if b is not alone, its bundle has threat at least $b > c$. This gives (C).
- $S = \emptyset$: a bundle with a good of R_i and at least two goods has a positive threat. This gives (E).

Conversely, each of (T), (BC), (B), (C), (E) describes situations shown to be safe above. Case (T) corresponds to the first and third items, and (BC) to $S = \{b, c\}$ or $S = \{a, b, c\}$. Case (B) forces $a \notin S$, so $S = \{b\}$ with a alone, or $S = \{b, c\}$; (C) forces $S = \{c\}$ with a and b alone; and (E) means that S is empty or a single alone good, which is (T), (B) or (C). □

Proposition 2 (limits of the shape). *Take values with* $v_i(a_i) > v_i(b_i) > v_i(c_i) > 0$ *and* $v_i(a_i) < v_i(b_i) + v_i(c_i)$ *for every agent.*

(a) Let agents 0, 1, 2 *rank* $g_0 \succ g_1 \succ p_i$*, where* p_i *is valued only by agent* i*. No EFX*$_0$ *allocation has all bundles of at most two goods, and* $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$*, to the three agents in any order, is EFX*$_0$*.*

(b) Let agents 0, 1, 2, 3 *rank* $g_0 \succ g_2 \succ p_0$*,* $g_0 \succ g_2 \succ p_1$*,* $g_1 \succ g_2 \succ p_2$ *and* $g_1 \succ g_2 \succ p_3$*, with* p_i *valued only by agent* i*. Every EFX*$_0$ *allocation has bundle sizes* $4, 1, 1, 1$*, and one exists.*

Proof. (a) Five goods in three bundles of at most two goods form bundles of sizes $2, 2, 1$, so exactly one good is alone. Two of the agents do not hold their top g_0; for them case (T) fails, and (C) and (E) need two or three alone goods. Hence both are in case (BC) or (B), and both hold g_1, which is impossible. In $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$ the agents are in cases (T), (B) and (C), respectively.

(b) For each of the tops g_0, g_1, some agent ranking it first does not hold it; call such an agent a *loser*. The two losers differ. A loser is safe only by (BC) or (B), which need it to hold g_2, or by (C) or (E), which need g_2 alone. The two losers cannot both hold g_2, so g_2 is alone, and nobody is in case (BC). A loser holding $\{g_2\}$ needs its top alone, and a loser not holding g_2 needs its top and g_2 alone. Hence g_0, g_1, g_2 are alone, held by three agents, and the fourth bundle is $\{p_0, p_1, p_2, p_3\}$. Such an allocation exists: agents 0, 1, 2, 3 receive g_0, g_2, g_1 and the four goods p_i, respectively, and are in cases (T), (B), (T) and (C). □

In (a) every agent values exactly three goods and is strictly balanced, so the one large bundle allowed by Theorem 1 cannot be avoided; by (b), its size cannot be bounded by three either.

E Related Work in a Table

An instance that no earlier result covers. Six agents value $\{g_0, g_1, g_6\}$, $\{g_0, g_1, g_7\}$, $\{g_0, g_2, g_8\}$, $\{g_2, g_3, g_4\}$, $\{g_3, g_5, g_9\}$ and $\{g_4, g_5, g_{10}\}$, with pairwise distinct values between 1 and 2. Then $n = 6$ and $m = 11$, and every agent values exactly three goods and is strictly balanced, so Theorem 1 applies. The other results of Table 1 do not, as we check row by row:

- $n = 3$ [5] and $n = 4$ [2]: here $n = 6$.
- At most three distinct valuations [7]: the six agents value different sets of goods, so the six valuations differ.

Table 1. Existence results for complete allocations under restrictions. *Version cited*: the published version where one exists, else arXiv. [a]Goods nobody values go to an arbitrary agent. [b]Includes additive. [c]Extended abstract with a proof sketch. [d]An AAMAS 2025 extended abstract has the degree and girth conditions. [e]Accepted at EC 2026.

Result	Restriction (agents, goods, who values what)	Notion	Complete	Valuations	Algorithm	Version cited
[5]	$n = 3$	EFX_0	yes	additive	pseudo-polynomial	JACM 2024
[7]	at most three distinct valuations	EFX_0	yes	additive	no bound stated	EC 2025
[12]	$m \leq n + 3$	EFX_0	yes	monotone	none stated	MOR 2024
[2]	$n = 4$, $m \leq 9$	EFX_0	yes	additive	none stated	arXiv
[4]	$m = n + 4$	EFX_0	yes, claimed	additive	none stated	Zenodo, not peer-reviewed
[6]	graphs: at most two valuers per good, two agents share at most one good	EFX_0	yes	monotone	polynomial time	MOR 2026 (EC 2023)
[1]	multigraphs: at most two valuers per good	EFX_0	yes	cancelable[b]	polynomial time	arXiv[e]
[13]	multigraphs: bipartite, or at most $\lceil n/4 \rceil - 1$ neighbours per agent, or girth at least six (non-parallel edges)	EFX_0	yes	monotone	not efficient (stated)	arXiv[d]
[8]	at most two valuers per good, $v_i(g) \in \{0, v_g\}$	EFX_0	yes	restricted additive	no time bound stated	WINE 2024
[9]	hypergraphs of girth at least four (a good joins its valuers)	EFX_0	yes	monotone	polynomial time	AAAI 2026
[14]	at most four relevant goods per agent	EFX	yes[a]	additive	polynomial time, announced	AAMAS 2024[c]
[11]	at most two relevant goods per agent	EFX_0	yes	additive; monotone	serial dictatorship; Lean-checked	GitHub, v1.2.0
this paper	at most three relevant goods per agent; any number of valuers per good	EFX_0	yes	additive	polynomial time, at most $4n$ trades	this paper

- $m \leq n + 3$ [12] and $m = n + 4$ [4]: here $m = n + 5$.
- At most two agents valuing each good [6,1,13,8] (a good is an edge between the agents that value it): three agents value g_0.
- Hypergraphs of girth at least four [9]: the first two agents both value g_0 and g_1, a cycle of length two.
- At most four relevant goods per agent [14]: this result covers the instance, but for EFX rather than EFX_0.
- At most two relevant goods per agent [11]: here three.

The script of Appendix F checks these properties and runs DE on 2,000 random choices of the values.

F Verification in Detail

Lean. The proofs of Sections 2–6 and of the appendices are machine-checked in core Lean 4 (`lean/EFX/K3DE*.lean` in [10], with `K3Extras.lean` for serial dictatorship and `RealValues.lean` and `K3Real.lean` for real values). They use no library beyond Lean's own and no unproved step, and every declaration rests only on Lean's three standard axioms; the repository's `lean/check.sh` checks this and also replays the library in a fresh kernel. DE itself is defined as a Lean function. To trust the main theorem, a reader needs to check the model (`lean/EFX/Model.lean`, about 50 lines, defining instances with natural-number values, allocations as maps from goods to agents, EFX_0 and the number of relevant goods) and the statement of `EFX.DE.de_correct`; Table 2 lists the theorem behind every result. The Lean files use other names for some notions: *needs* for wants, *junk* for leftover goods, *utility* for score, *exposed* for blocker, *protecting good* for the leftover good h_x, *need arcs* and *exposure arcs* for want and pair arrows (Lean's arcs point the way goods move, as σ does, so they are the arrows here reversed), *need cycle* and *exchange cycle* for rings, *pair chain* for a chain, (P) for (NC), and *valid absorber* for an agent that can finish (a notion that also allows a pair holder to finish, as when every agent holds its pair). A single Lean lemma covers both chains and rings, by closing a chain with an arc from its last agent back to x.

Real values. The model has natural-number values, and real values are reduced to it by an encoding. Let agent i have relevant goods of values $x_1, x_2, x_3 > 0$. (With one or two relevant goods, the missing slots repeat the first good in these comparisons, so some compared sums count a good twice; no set of goods contains a missing slot.) After the goods common to both sides are cancelled, a comparison of two sums of its

Table 2. The results of this paper and their Lean theorems in [10], in the namespace `EFX.DE` unless the name starts with `EFX.`; each has a `#print axioms` certificate in its file. *Rankings*: stated for every profile of rankings, over DE's states. $\mathbb{N}$: natural-number values, those of the model.

Result	Lean theorem	Values; what is proved
Theorems 1 and 5; Theorem 2 without the time	`de_correct`	$\mathbb{N}$: DE's output is EFX_0, all bundles but at most one have at most two goods, at most n peeling rounds and $4n$ trades
The same, real values	`deOrd_correct`, `thmD_ordered`	any linearly ordered cancellative commutative monoid, such as $\mathbb{R}_{\geq 0}$; nonnegative values
Running time (Theorem 2)	`de_eq_spec`, `deC_cost`	$\mathbb{N}$: a counted program returns DE's output within $750(n+1)(n+m+1)$ units
The same, real values	`deOrdC_cost`, `EFX.K3.surrogateC_cost`	as above, plus the encoding: at most $n(m+12)$ comparisons and $10nm+971n$ other units
The encoding of real values	`EFX.K3.agree_surrogate`, `EFX.efx0_iff_of_agree`	ordered values
Lemmas 1 and 2; rule R1 and serial dictatorship for two goods	`peeling`, `R1fail`, `r1_of_two`, `EFX.Inst.serialDict_efx0`	$\mathbb{N}$
Lemmas 3 and 4	`draft_valid`, `transfer`	rankings
Theorem 3	`soundness`	$\mathbb{N}$; every valuation consistent with the rankings (balance $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$ suffices)
Lemmas 5 and 6	`exchange`, `exchange_scores`	rankings; one lemma for both
Lemma 7	`absorber_iff`, `not_exposed_of_none`	rankings
Theorem 4, Corollary 1	`improvement`, `completable_of_undominated`	rankings
Algorithm 1, the loop	`loop_spec`, `reps_first_blocker`, `step_next_scores`	rankings
The examples of Sections 1 and 6, Appendix B	`Remarks.efx_not_efx0_intro`, `Examples.small_spec`, `Examples.worked_spec`, `ShortExamples.w_exactly_four`, `ShortExamples.w_completions`, `Remarks.w_cycles`	by evaluation
Proposition 1; the ring family	`prop_short`, `ShortExamples.w_attains`, `ShortExamples.s8_attains`, `Rings.nExp_eq`, `Rings.no_free_absorber_iff`	rankings; every k and d
Lemma 8, Proposition 2	`Limits.safe_iff_cases`, `Limits.limits_a`, `Limits.limits_b`, `Limits.limits_a_ordered`, `Limits.limits_b_ordered`	$\mathbb{N}$, every value profile of the stated form; Proposition 2 also for ordered values

values becomes a comparison of two disjoint sets of these goods, and since the values are positive, it is decided by twelve basic comparisons: $x_k \le x_l$ for $k \ne l$, and $x_k \le x_l + x_r$ and $x_l + x_r \le x_k$ for $\{k, l, r\} = \{1, 2, 3\}$. Both directions are asked, so that ties are recorded. Using an oracle for $\le$, a program first asks the m questions $v_i(g) \le 0$, which find the relevant goods, and then these twelve questions. It gives the three goods the first of 31 triples of natural numbers with the same twelve answers, and every other good keeps the value 0; Lean's kernel checks that this table of triples covers every consistent pattern of answers. The new values w answer every comparison of two sums of an agent's values as v does (`EFX.K3.agree_surrogate`). This suffices, because the EFX_0 condition $v_i(X_i) \ge v_i(X_j \setminus \{h\})$ compares two such sums; hence an allocation is EFX_0 for w if and only if it is EFX_0 for v, and the relevant goods are the same (`EFX.efx0_iff_of_agree`, `EFX.numRelevant_eq_of_agree`). Consequently, DE run on w returns an allocation that is EFX_0 for v, of the same shape and within the same bounds (`EFX.DE.deOrd_correct`).

Running time. A counted program, proved to return exactly DE's output (`de_eq_spec`), takes at most $750(n+1)(n+m+1)$ units, including reading the input (`deC_cost`). As in a random-access machine, each elementary operation, comparison, array read and array write costs one unit, and arrays are used single-threaded; a natural number costs one unit whatever its size. On real values, the encoding adds at most $n(m+12)$ comparisons and $10nm+971n$ other units (`EFX.K3.surrogateC_cost`, `deOrdC_cost`).

What is not formalized. (1) That the units match the steps of a machine, a property of the counted program checked by reading it. (2) That the nonnegative reals satisfy the axioms of Lean's class of values. Core Lean has no real numbers, so this textbook fact is assumed (the instances for natural numbers and integers are checked). (3) That DE run directly on real values makes the same decisions as on the encoded values. This is true because every decision is a comparison that the encoding preserves, but Lean states the theorem for DE run on w. (4) That the Lean statements say what the text says. They were written in AI agent sessions of Claude Code, the coding assistant that helped with this work, and compared with the text by independent AI sessions (`k3/simplify/po/referee/lean_audit*.md` in [10]), not by a human; the model and the statement of `de_correct` are short enough to check by hand. (5) The second proof mentioned below, the novelty claim, and the tests.

Tests. The tests are evidence, not proof. Three implementations, each asserting every claim of a proof and checking its outputs against the raw definition of EFX_0 in exact arithmetic, ran without failure on every profile with $n = 2$ and $m \leq 6$ or with $n = 3$ and $m \leq 7$, on part of the profiles with $n = 4$ and $m \leq 6$, on core instances with $n = 5, 6$, and on random profiles with up to 9 agents. The counts are about 1.7 million valid states for the proof of Section 5, about 15.5 million for an independently found second proof (a walk instead of a choice of distinct leftover goods), and about 2.9 million runs of a referee's implementation written from the proof notes alone (`k3/simplify/po/` in [10]). The first and third implementations run DE's loop with the same three steps (up to the choice among leftover goods); the second tests the lemmas of Sections 4 and 5 on its own loop, which lets a free agent holding nothing finish first. The script `paper/k3-simple/examples/check_examples.py`, written from this paper with the loop as stated here, recomputes every example and every computed number in the paper and runs DE, peeling included, on random instances with ties, unbalanced agents and goods that nobody values.